

데모 실행 매뉴얼

서버 기동 · 보이스피싱 의심통화 탐지 · 백그라운드 이상거래 경보

대상	2026 금융 AI Challenge — AI_Finance_Sec
버전	커밋 cc3f126 기준
작성일	2026-08-30
실행 환경	Windows 11 · PowerShell
검증 상태	본 문서의 전 절차·전 수치 실측 완료

이 문서의 모든 화면 값과 명령 출력은 실제 실행 결과입니다. 서버를 완전히 내린 상태에서 1장부터 순서대로 재실행하여 확인한 값이며, 임의로 작성한 예시가 아닙니다. 검증 이력은 8장에 있습니다.

1. 사전 준비

1.1 필요한 것

항목	필수 여부	용도
Node.js	필수	프론트(Next) 서버
Python 가상환경 backend\.venv	전체 스택만	FastAPI + LangGraph 백엔드
Ollama + qwen2.5:3b	전체 스택만	로컬 경량 모델 추론
브라우저	필수	Chrome 또는 Edge

Mock 모드만 쓸 거라면 Node.js 하나면 됩니다.
경보 티어, 이상거래 신호, 3블록 문구, 피해대응 액션까지 전부 Mock 모드에서 재현됩니다. Python-Ollama는 **실제 LangGraph 추론 경로**를 보여줄 때만 필요합니다.

1.2 프로젝트 경로

```
C:\Users\SOCISOFT\AI_fiannce_sec\AI_Hacker\AI_Finance_Sec
```

이하 모든 명령은 이 경로에서 실행합니다.

주의 — 프로덕션 서버로는 시연하지 마십시오.
Windows에서 `npm run start:win` 은 하위 디렉터리 정적 자산을 전부 404로 응답합니다. CSS:JS가 붙지 않아 화면이 통째로 깨지고 버튼도 동작하지 않습니다. 의존성(vinext)의 Windows 경로 처리 버그이며, 시연은 반드시 `npm run dev:win` 또는 `start-demo.ps1` 로 하십시오. 상세는 9.3절.

2. 서버 실행 절차

2.1 방법 A — 간편 실행 (Mock 모드, 1분)

1. PowerShell을 열고 프로젝트 경로로 이동합니다.

```
cd C:\Users\SOCISOFT\AI_fiannce_sec\AI_Hacker\AI_Finance_Sec
```

2. 개발 서버를 실행합니다.

```
npm run dev:win
```

3. 다음 줄이 출력될 때까지 기다립니다. 첫 실행은 약 20~30초 걸립니다.

```
→ Local: http://localhost:3000/
```

4. 브라우저에서 `http://localhost:3000` 에 접속합니다.

반드시 localhost 로 접속하십시오.

개발 서버는 IPv6 주소([::1]:3000)에만 바인딩됩니다. 127.0.0.1:3000 으로는 연결되지 않습니다.

확인 기준 — 화면 상단에 AI_Finance_Sec 로고와 샘플 처리 과정 9단계 파이프라인이 보이고, 우상단 모델 표시가 Mock 이면 정상입니다.

2.2 방법 B — 전체 스택 실행 (실제 LangGraph 경로)

1. Ollama가 떠 있는지 확인합니다. 모델 목록이 출력되어야 합니다.

```
curl.exe http://127.0.0.1:11434/api/tags
```

기대: qwen2.5:3b 가 목록에 포함

없으면 내려받습니다. ollama pull qwen2.5:3b

2. FastAPI 백엔드를 실행합니다. (새 PowerShell 창)

```
cd C:\Users\SOCISOFT\AI_fiannce_sec\AI_Hacker\AI_Finance_Sec
$env:OLLAMA_MODEL = "qwen2.5:3b"
.\backend\.venv\Scripts\python.exe -m uvicorn backend.main:app --host 127.0.0.1 --port 8000
```

3. 백엔드 상태를 확인합니다. (또 다른 창)

```
curl.exe http://127.0.0.1:8000/health
```

실측 출력:

```
{"status":"ok","provider":"ollama","model":"qwen2.5:3b",
  "ollama_base_url":"http://127.0.0.1:11434","memory":"demo-only-in-memory"}
```

4. 프론트를 실행합니다. 2.1절과 동일하게 npm run dev:win

5. 브라우저 우상단 설정 → Ollama Local 선택 → 적용

한 번에 띄우려면

```
powershell -ExecutionPolicy Bypass -File .\scripts\start-demo.ps1
```

Ollama-FastAPI-Next를 함께 기동하고 Cloudflare Quick Tunnel로 공개 URL을 발급합니다. 공개 URL이 생기므로 시연이 끝나면 반드시 .\scripts\stop-demo.ps1 로 종료하십시오.

기동 소요시간 실측 — FastAPI 즉시 응답, Next 약 20초. 두 서버 모두 200 이 나오면 준비 완료입니다.

3. 시나리오 A — 보이스피싱 의심 통화 탐지 (프론트)

통화 문장이 한 줄씩 쌓이면서 위험도가 어떻게 올라가고, 그에 따라 경보가 어떤 단계로 상승하는지 확인하는 절차입니다. **거래 정보 없이 대화 맥락만으로** 판정하는 경로를 봅니다.

3.1 실행 절차

- 1. 좌측 시나리오 목록에서 **01 검찰·금감원 사칭** 을 클릭합니다.
- 2. ▶ **샘플 분석 시작** 버튼을 클릭합니다.
- 3. 중앙 패널에 통화 문장이 약 1.25초 간격으로 한 줄씩 추가됩니다.
- 4. 문장이 추가될 때마다 **우측 패널의 세 영역**을 함께 관찰합니다.
 - **통합 위험도** — 점수와 등급
 - **경보 전달 계획** — 티어와 채널
 - **고객에게 보이는 문구** — 3블록 안내문

3.2 단계별 기대 결과 (실측값)

시점	위험도	등급	이상거래 배지	경보 티어	문구패널
재생 전	0	안전	대기	T0 · 무개입	미표시
1번째 대사	24	안전	대기	T0 · 무개입	미표시
2번째 대사	45	주의	대기	T1 · 인라인 배너	표시
3번째 대사	67	주의	합계 42 · 반영 30	T3 · 다채널 경보	표시
4번째 대사	100	위험	합계 42 · 반영 30	T3 · 다채널 경보	표시

3번째 대사에서 등급은 '주의'인데 티어가 T3로 뛰는 이유

통화 축(67점 → 2단계)과 거래 축(원점수 42 → 2단계)이 **동시에 반응했기** 때문입니다. 두 축이 모두 걸리면 한 단계 상승하는 규칙이 적용되어 2 → 3이 됩니다. 통화만 의심스럽거나 거래만 이상한 경우보다, 둘이 겹칠 때 실제 피해 확률이 훨씬 높기 때문입니다.

3.3 최종 화면에서 확인할 항목

영역	확인 내용
전달 채널	인앱 인터스티셜 · ARS 콜백
억제 채널	푸시 알림 · SMS — 통화 중이므로 억제
강제 체류	30초, 은밀 모드 · 화면 전환 없음
신뢰인 알림	발송
고객 문구 ①	관측된 이상 3건
고객 문구 ②	[기관 사칭] 보호 명목 계좌 이동 요구
고객 문구 ③	통화 즉시 종료 — 단일 행동만 제시
셀프체크	3문항 노출
피해대응 액션	☎ 1394 피해 신고·상담 / 📞 지급정지 절차

3.4 피해대응 액션 동작 확인

- 1. 채팅 영역 하단의 ☎ 1394 피해 신고·상담 버튼을 클릭합니다.
- 2. 화면 하단에 안내 토스트가 뜹니다. 실측 문구:

1394 신고·상담 절차를 시뮬레이션했습니다. 긴급한 상황은 112에 신고해야 하며 실제 신고는 수행되지 않습니다.

- 3. 📞 지급정지 절차 버튼도 클릭합니다.

해당 금융회사 지급정지 요청 절차를 시뮬레이션했습니다.

이 데모는 실제 외부 조치를 수행하지 않습니다.
1394 신고, 112 신고, 지급정지 요청 모두 안내만 하며 실행하지 않습니다. API 응답의 external_action_executed 값은 항상 false 입니다. 시연 중 이 점을 반드시 함께 설명하십시오.

3.5 오탐 확인 — 정상 고객에게는 개입하지 않는다

- 1. 좌측에서 04 정상 은행 상담 (Hard Negative) 을 선택합니다.
- 2. ▶ 샘플 분석 시작 을 클릭하고 끝까지 재생합니다.

합격 기준 (실측) — 위험도 18 / 안전, 이상거래 합계 0 · 반영 0, 경보 T0 · 무개입. 전달 채널 없음, 고객 문구 패널 미표시, 액션 버튼 없음.
정상 고객이 겪는 마찰이 0이라는 점이 이 시나리오의 핵심입니다. “송금 화면에 위험 단어가 있으면 무조건 경고” 하는 방식과의 차이를 보여줍니다.

4. 시나리오 B — 백그라운드 이상거래 탐지

통화 내용과 **무관하게** 거래 자체만 관찰해 이상 신호를 만드는 경로입니다. 평소 거래 프로필과 이번 이체를 대조합니다.

4.1 판정 기준

평소 거래 프로필 (기준선)

항목	값
중앙값 이체액	120,000원
상위 5% 이체액	500,000원
등록 수취인	KB-1002-3355 , SHINHAN-110-4477 , NH-352-9910
평소 이체 시간대	08시 ~ 22시

이상 신호 8종

신호	가중치	판정 조건
평소 상위 5% 이체액의 2배 초과	+12	금액 > 1,000,000원
평소 상위 5% 이체액 초과	+8	금액 > 500,000원
처음 보내는 수취인 계좌	+10	등록 수취인 목록에 없음
평소 이체하지 않는 시간대	+6	08시 이전 또는 22시 이후
통화 중 이체 시도	+12	통화 연결 상태
신규 기기.앱 설치 직후 이체	+10	설치 24시간 이내
짧은 시간 내 분할 이체 반복	+8	최근 1시간 2건 이상
이체 한도 상향 직후	+8	상향 24시간 이내

합계와 반영값이 다른 이유

신호 합계는 **상한 30**으로 잘라 위험 점수에 더합니다. 거래 신호가 통화 맥락 판정을 압도하지 못하게 하려는 제한입니다. 다만 **경보 티어 판정에는 상한 전 원점수**를 씁니다. 그래서 화면 배지가 **합계 42 · 반영 30** 으로 두 값을 함께 보여줍니다.

4.2 화면에서 확인하는 절차

1. **01 검찰·금감원 사칭** 을 선택하고 ▶ **샘플 분석 시작** 을 클릭합니다.
2. 1~2번째 대사 동안 **백그라운드 이상거래 신호** 배지는 **대기** 입니다. 아직 이체가 발생하기 전이기 때문입니다.
3. **3번째 대사부터** 배지가 **합계 42 · 반영 30** 으로 바뀌고 아래에 신호 4건이 나타납니다.

화면에 표시되는 신호	값	가중치
평소 상위 5% 이체액의 2배 초과	9,800,000원 · 평소 500,000원	+12
처음 보내는 수취인 계좌	WOORI-777-0001	+10
통화 중 이체 시도	통화 연결 상태	+12
이체 한도 상향 직후	24시간 이내 상향	+8
합계		42

4.3 경보 티어 결정 규칙

티어	조건	전달 채널	마찰
T0 무개입	통화 <40 · 거래 <18	없음	0초
T1 인라인 배너	한 축만 반응	인앱 인라인 배너	0초
T2 인터스티셜	통화 60+ 또는 거래 30+	인앱 인터스티셜 · 푸시	15초 + 셀프체크 3문항
T3 다채널 경보	통화 75+ 또는 이미 송금	인앱 · ARS 콜백 · SMS · 신뢰인	30초

- 두 축이 모두 반응하면 **한 단계** 상승합니다.
- 이미 송금한 경우는 조건과 무관하게 **무조건 T3** 입니다.
- **통화 중이면 푸시·SMS를 억제**하고 ARS 콜백과 은밀 모드로 대체합니다.

통화 중 푸시를 막는 이유

보이스피싱은 범인이 “은행에서 연락 오면 이렇게 답하세요” 라고 미리 코칭해 둡니다. 통화 중 푸시 알림은 범인이 피해자 화면을 함께 보게 만들어 오히려 대응을 무력화시킵니다. 그래서 통화 중에는 화면 전환이 없는 은밀 모드로 두고, **통화를 물리적으로 끊는 ARS 콜백**을 사용합니다.

4.4 신호를 직접 바꿔 시험하기

시나리오의 거래 조건은 `app\lib\engine.ts` 의 `transactionContext` 에 있습니다.

```
transactionContext: { amount: 9_800_000, payee: "WOORI-777-0001",
                      hour: 21, inCall: true, limitRaised: true },
```

값을 바꾸고 저장하면 개발 서버가 자동 반영합니다.

바꿀 값	기대 변화
<code>inCall: false</code>	억제 채널이 사라지고 푸시 알림 · SMS 가 전달 채널로 이동
<code>payee: "KB-1002-3355"</code>	처음 보내는 수취인 신호(+10) 사라짐
<code>amount: 150_000</code>	금액 초과 신호(+12) 사라짐
<code>hour: 3</code>	평소 이체하지 않는 시간대(+6) 추가
전부 정상값으로	합계 0 → T0 무개입

값을 바꾼 뒤에는 반드시 회귀 테스트를 돌려주세요.

`npm run test:win` 의 `scenario transaction contexts keep the fused score unchanged` 항목이 깨지지 않아야 합니다. 사기 시나리오는 반영 점수 30, 정상 시나리오는 0을 유지해야 기존 평가 지표가 보존됩니다.

5. API 직접 검증 (화면을 거치지 않는 확인)

백엔드가 실제로 계산했는지 가장 확실하게 확인하는 방법입니다. 2.2절이 완료된 상태여야 합니다.

5.1 실행

1. 요청 본문 파일을 만듭니다. 한글이 포함되므로 반드시 UTF-8로 저장해야 합니다.

```
cd C:\Users\SOCISOFT\AI_fiannce_sec\AI_Hacker\AI_Finance_Sec
.\backend\.venv\Scripts\python.exe -c "import json,io; io.open('req.json','w',encoding='utf-8').write(json.dumps({'message':'서울중앙지검 수사관입니다. 지금 즉시 안전계좌로 이체하세요.','session_id':'manual-check','transaction':{'amount':9800000,'payee':'WOORI-777-0001','hour':21,'in_call':True,'limit_raised':True}},ensure_ascii=False))"
```

2. API를 호출합니다. 로컬 모델 추론이 두 번 일어나므로 약 45~50초 걸립니다.

```
curl.exe -s -X POST http://127.0.0.1:8000/api/chat `
-H "content-type: application/json" `
--data-binary "@req.json" -o resp.json -w "http:%{http_code}`n"
```

3. 응답을 확인합니다.

```
$env:PYTHONIOENCODING="utf-8"
.\backend\.venv\Scripts\python.exe -c "import json,io; d=json.load(io.open('resp.json',encoding='utf-8')); print('trace  :',' -> '.join(d['trace'])); print('anomaly :',d['transaction_anomaly']['raw_score'],'->',d['transaction_anomaly']['score']); print('fusion  :',d['risk']['fusion'],'| tx',d['risk']['transaction_score']); a=d['alert']; print('alert   : T%s %s'%(a['tier'],a['tier_name'])); print('channels:',a['channels']); print('suppress:',a['suppressed_channels']); print('cta     :',a['message']['primary_action']); print('external:',d['external_action_executed'])"
```

5.2 실측 출력

```
trace   : prepare_input -> transaction_signal_agent -> structured_risk_agent
         -> conservative_fusion -> knowledge_agent -> policy_agent
         -> ollama_generate -> policy_guardrail -> safety_verifier
         -> alert_planner -> finalize
anomaly : 42 -> 30
fusion  : rule+structured+transaction | tx 30
alert   : T3 다채널 경보
channels: ['인앱 인터스티셜', 'ARS 콜백']
suppress: ['푸시 알림', 'SMS']
cta     : 통화 즉시 종료
external: False
```


5.3 합격 판정 기준

확인 항목	기대값
trace	transaction_signal_agent 와 alert_planner 가 둘 다 포함
anomaly	42 -> 30
fusion	rule+structured+transaction
alert	T3 다채널 경보
suppress	['푸시 알림', 'SMS'] — 통화 중 억제 동작 증명
external	False — 외부 조치 미실행

5.4 대조군 — 거래 컨텍스트 없이 호출

transaction 필드를 빼고 같은 절차를 반복하면, 거래 신호가 기존 판정을 오염시키지 않는다는 것을 확인할 수 있습니다.

```
anomaly : 0 -> 0
fusion  : rule+structured      <- "+transaction" 이 붙지 않음
score   : 100                  <- 통화 맥락만으로 낸 점수 그대로
alert   : T3 다채널 경보
channels: ['인앱 인터스티셜', '푸시 알림', 'ARS 콜백', 'SMS']
suppress: []                   <- 통화 중이 아니므로 억제 없음
```

이 대조가 증명하는 것

- ① 거래 컨텍스트가 없으면 기존 위험 판정이 한 글자도 바뀌지 않는다 — 기존 평가 지표가 보존됩니다.
- ② 통화 중이 아니면 푸시·SMS가 정상적으로 전달 채널에 포함된다 — 억제 로직이 무조건 막는 것이 아니라 통화 상태에만 반응합니다.

6. 자동 테스트 (서버 없이 확인)

1. 백엔드 회귀 테스트

```
cd C:\Users\SOCISOFT\AI_fiannce_sec\AI_Hacker\AI_Finance_Sec
$env:PYTHONIOENCODING="utf-8"
.\backend\.venv\Scripts\python.exe -m pytest backend\tests -q
```

기대: 40 passed

2. 프론트 회귀 테스트 (빌드 포함, 1~2분)

```
npm run test:win
```

기대: pass 20 / fail 0

3. 이상거래·경보 관련 테스트만 보려면

```
.\backend\.venv\Scripts\python.exe -m pytest backend\tests -q -k "transaction or alert" -v
```

4. 정적 분석

```
npm run lint
```

이 테스트들이 보장하는 것

- TypeScript 엔진과 Python 엔진의 **30조합 전수 교차 비교** (거래 컨텍스트 6종 × 위험 5종)
- 통화 중 푸시·SMS 억제 및 ARS 콜백 사용
- 단일 CTA가 정책 허용목록을 벗어나지 않음
- `external_action_executed == false` — 허위 조치 주장 없음
- 정상 고객 T0 무개입
- 거래 컨텍스트가 없을 때 기존 점수 불변

회귀 테스트는 프로덕션 서버 버그의 영향을 받지 않습니다.

빌드 산출물을 worker로 직접 import해 검증하므로, 9.3절의 정적 자산 404 문제와 무관하게 정상 동작합니다.

7. 종료 절차

1. 각 PowerShell 창에서 **Ctrl + C** 를 누릅니다.
2. `start-demo.ps1` 로 띄웠다면 정지 스크립트를 실행합니다.

```
.\scripts\stop-demo.ps1
```

3. 창이 닫혔는데 포트가 남아 있으면 직접 종료합니다.

```
netstat -ano | findstr ":3000 :8000"  
taskkill /PID <PID> /T /F
```

공개 URL을 발급했다면 반드시 종료하십시오.

`start-demo.ps1` 은 Cloudflare Quick Tunnel로 외부 접근 가능한 https 주소를 만듭니다. 시연 후 종료하지 않으면 계속 접근 가능한 상태로 남습니다.

8. 검증 이력

본 문서 작성 시점에 모든 서버를 완전히 내린 뒤(콜드 스타트), 1장부터 순서대로 재실행하여 전 절차를 확인했습니다.

검증 항목	결과	비고
2.2 FastAPI 기동	통과	health 200 즉시 응답
2.1 Next dev 기동	통과	약 20초 후 200
3.2 단계별 위험도 등급	통과	0 → 24 → 45 → 67 → 100, T0→T1→T3
3.3 T3 화면 항목	통과	채널·억제·체류·신뢰인·3블록·셀프체크 전부 표시
3.4 피해대응 액션	통과	버튼 2종 클릭 및 토스트 문구 확인
3.5 정상 고객 무개입	통과	18 / 안전, T0, 마찰 0
4.2 이상거래 신호 4건	통과	합계 42 · 반영 30
5.2 API 실측 출력	통과	trace 11단계, 응답 47.5초
5.4 대조군	통과	fusion=rule+structured, 점수 불변
6.1 백엔드 테스트	통과	40 passed
6.2 프론트 테스트	통과	pass 20 / fail 0
브라우저 콘솔 에러	0건	Chrome 자동 조작 15개 항목 전부 통과

9. 문제 해결

9.1 자주 발생하는 문제

증상	원인 및 조치
화면이 통째로 깨져 보임	npm run start:win 을 사용한 경우입니다. npm run dev:win 으로 실행하십시오. 9.3절 참조
브라우저에서 연결되지 않음	127.0.0.1:3000 이 아니라 localhost:3000 으로 접속
포트가 이미 사용 중 (EADDRINUSE)	netstat -ano findstr ":3000" 로 PID 확인 후 taskkill /PID <PID> /T /F
이상거래 신호가 계속 '대기'	아직 3번째 대사 전입니다. 재생을 더 진행하십시오
API가 400 error parsing the body	요청 본문이 UTF-8이 아닙니다. 5.1절처럼 파일로 저장한 뒤 --data-binary 사용
pytest 출력의 한글이 깨짐	\$env:PYTHONIOENCODING="utf-8" 를 먼저 실행
Ollama 응답 없음 / 타임아웃	Ollama 미실행이거나 모델 미설치입니다. ollama pull qwen2.5:3b
Ollama 응답이 너무 느림	정상입니다. 구조화 판정 + 생성 2회 호출로 45~50초 소요됩니다

9.2 첫 실행이 느린 경우

`npm run dev:win` 첫 실행 시 Vite가 의존성을 재최적화하므로 20~30초 걸립니다. `Re-optimizing dependencies` 메시지가 보이면 정상이며, 그대로 기다리면 됩니다.

9.3 프로덕션 서버 정적 자산 404 (미해결 · 의존성 버그)

Windows에서 `vinext start` 는 하위 디렉터리의 정적 자산을 전부 404로 응답합니다. `/og.png` 같은 루트 파일은 200 이지만 `/assets/*.js`, `/assets/*.css`, `/data/*.json` 이 모두 404가 되어 CSS-JS가 불지 않습니다.

원인은 의존성 `vinext` 의 Windows 경로 처리입니다.

```
// node_modules/vinext/dist/server/static-file-cache.js
relativePath: path.relative(base, fullPath) // Windows: "assets\index.js"
const pathname = "/" + relativePath; // "/assets\index.js" 로 색인됨
```

요청 경로는 `/assets/index.js` (슬래시)이므로 영원히 매칭되지 않습니다. 구분자가 없는 루트 파일만 우연히 맞아떨어 집니다. macOS·Linux에서는 `path.relative` 가 슬래시를 반환하므로 발생하지 않습니다.

`node_modules` 직접 수정은 재설치 시 사라지고 형상관리도 되지 않아 적용하지 않았습니다. **시연은 `dev:win` 또는 `start-demo.ps1` 로 하십시오.**

AI_Finance_Sec 데모 실행 매뉴얼 · 2026-08-30 작성 · 커밋 cc3f126 기준

본 문서의 모든 수치는 실제 실행 결과입니다. 상세 설계 근거는 `docs/작업로그_백그라운드_이상거래_경보.md` 를 참조하십시오.

데모의 통화·거래 데이터는 전부 합성 데이터이며, 외부 신고·지급정지는 실제로 수행되지 않습니다.